

高级语言程序设计

实验报告

南开大学 计算机大类

姓名 苗诗卉

学号 2511972

班级 模拟 2-1

2026 年 5 月 10 日

目录

高级语言程序设计大作业实验报告

一. 作业题目

二. 开发软件

三. 课题要求

四. 主要流程

1. 整体流程

2. 算法或公式

3. 单元测试

五. 单元测试

六.AI 工具使用披露

七. 收获

高级语言程序设计大作业实验报告

一. 作业题目

基于 SFML 的 2D 射击小游戏（Kitty Sky）设计与实现

二. 开发软件

开发环境：Visual Studio 2022

编程语言：C++

图形库：SFML 2.5.1（用于窗口渲染、图形绘制、音频播放、输入处理）

构建工具：CMake

三. 课题要求

1. 采用**面向对象编程**思想，完成游戏模块化设计；
2. 实现完整的 2D 射击游戏核心功能（玩家控制、敌人生成、子弹发射、碰撞检测、游戏状态管理）；
3. 实现游戏特效、音效、道具系统，提升游戏体验；
4. 完成代码功能测试，保证程序稳定运行；
5. 规范代码结构，具备可读性与可维护性。

四. 主要流程

1. 整体流程

实现思路：

采用状态机模式管理游戏全流程，分为菜单状态、游戏状态、暂停状态、游戏结束状态；

定义抽象基类 Entity（游戏实体基类），封装位置、移动、渲染等通用属性与方法，玩家、敌人、子弹、道具均继承该类实现多态；

核心架构：

1. 基类：Entity（抽象实体类）、State（抽象状态类）；
2. 子类：Player、Enemy、Bullet、Pickup 继承自 Entity；GameState、MenuState 继承自 State；
3. 辅助模块：命令模式处理输入、粒子系统实现特效、音乐播放器管理音效；
4. 运行流程：启动程序→菜单界面→开始游戏→操控射击 / 收集道具→生命值耗尽→游戏结束→返回菜单。

2. 算法或公式

（1）玩家移动算法

基于键盘输入与帧时间平滑移动，限制玩家在窗口范围内：

移动公式：位置偏移量 = 移动方向 × 速度 × 帧时间

边界检测：通过实体矩形边界判断，防止移出窗口。

（2）碰撞检测算法

本项目采用 AABB 轴对齐矩形碰撞检测，基于 SFML 内置函数 `sf::FloatRect::intersects()` 实现。

A. 原理

游戏中所有实体（玩家、敌人、子弹、道具）均使用矩形包围盒包裹自身纹理，矩形的位置和大小会跟随实体实时更新。当两个矩形的区域产生重叠时，判定为发生碰撞。

B. 实现方法

- 调用 `getGlobalBounds()` 获取实体的实时矩形区域；
- 使用 `rect1.intersects(rect2)` 判断是否重叠；

- 返回 true 表示碰撞，false 表示未碰撞

C. 碰撞分组与逻辑

- 子弹 ↔ 敌人：敌人扣血 / 销毁，子弹消失，播放爆炸特效，增加分数
- 玩家 ↔ 敌人：玩家扣血，敌人销毁，播放受伤音效
- 玩家 ↔ 道具：玩家获得强化（加血、加子弹、加攻速），道具消失

D. 核心代码逻辑

```
sf::FloatRect playerBounds = player.getGlobalBounds();

sf::FloatRect enemyBounds = enemy.getGlobalBounds();

if (playerBounds.intersects(enemyBounds)) {

    // 执行受伤逻辑：玩家减血、敌人销毁

}
```

(3) 敌人/子弹/道具生成算法

A. 原理

使用 sf::Clock 或累计帧时间 dt 记录时间流逝，当累计时间达到设定的生成间隔时，执行一次生成逻辑，并重置计时器。

B. 实现方法

- 定义生成间隔：敌人 1.0 秒生成一个，道具 5 秒生成一个
- 每帧累加 deltaTime 时间
- 时间达标 → 生成实体 → 重置计时器
- 随机生成位置：水平方向随机，垂直方向从屏幕顶部出现

C. 优势

- 帧率不影响生成速度，游戏流畅稳定
- 可动态调整生成频率，实现难度递增
- 控制实体数量，避免内存占用过高

D. 核心代码逻辑

```
spawnTimer += dt.asSeconds();

if (spawnTimer >= 1.0f) {
    spawnEnemy();      // 生成敌人
    spawnTimer = 0.0f; // 重置计时器
}
```

(4) 视差背景滚动算法

采用 多层背景 + 不同滚动速度 实现纵深感，是经典 2D 飞行射击 / 卷轴游戏的视觉效果方案。

A. 原理

将背景分为两层：远景层（慢速）、近景层（快速）。两层同时向下滚动，速度不同，形成飞行前进的视觉效果。

B. 实现方法

- 两层背景图片大小与屏幕一致
- 近景层速度 = 基础速度
- 远景层速度 = 基础速度 $\times$ 0.5（更慢，显得更远）
- 当背景滚动超过图片高度时，重置 Y 坐标，实现无缝循环

C. 视觉效果

- 滚动流畅无断层
- 空间感强，提升游戏沉浸感

D. 核心代码逻辑

```
// 近景快速滚动
bgLayer1Y += scrollSpeed * dt.asSeconds();

// 远景慢速滚动
bgLayer2Y += scrollSpeed * 0.5f * dt.asSeconds();

// 超出高度后重置，实现无缝滚动
if (bgLayer1Y > textureHeight) {
    bgLayer1Y = 0;
}
```

3. 单元测试

针对核心模块设计用例：

- 1. 实体类测试：测试玩家、敌人、子弹的创建、移动、渲染功能；
- 2. 碰撞检测测试：验证子弹击中敌人、玩家触碰道具 / 敌人的判定逻辑；
- 3. 状态机测试：测试菜单、游戏、暂停、结束状态的切换功能；
- 4. 输入控制测试：验证键盘按键控制玩家移动、发射子弹、暂停游戏的响应；
- 5. 特效与音效测试：验证爆炸粒子、Bloom 光影、背景音乐 / 音效正常播放。

五. 单元测试

测试案例定义

表 1：测试案例

表格

输入 / 操作	输出 / 结果	目的
运行程序	正常显示游戏菜单界面	验证程序启动与菜单初始化
按下 WASD	玩家平滑移动	验证玩家控制功能
按下空格	发射子弹、播放音效	验证发射与音效功能
子弹命中敌人	敌人销毁、播放爆炸特效	验证碰撞与销毁逻辑
触碰道具	获得强化 / 恢复生命值	验证道具拾取功能
按下 ESC	进入暂停界面	验证状态切换功能
生命值归 0	进入游戏结束界面	验证结束逻辑

测试结果

- 1. 全部用例通过，程序稳定无崩溃、无卡顿；
- 2. 初始敌人生成过密，调整时间间隔后修复；
- 3. 偶现碰撞失效，优化边界计算后解决；
- 4. 画面、音效、控制响应均符合预期。

六. AI 工具使用披露

1. AI 工具名称及版本

2. 具体用途

1. 技术学习与选型：学习 C++ 第三方库的对比与选择、学习 GitHub/Gitee 代码托管平台的使用规范、提交方法、分支管理；
2. 美术资源生成：生成游戏所需的角色、敌人、道具、背景等图片美术素材设计方案；
3. 代码调试：解决程序崩溃、内存泄漏、输入响应异常、资源加载失败等问题；

3. 使用位置

1. 技术学习：GitHub/Gitee 使用教程、C++ 游戏开发库学习与选型文档；
2. 美术资源：游戏角色、敌人、道具、UI 界面的图片素材设计与生成；

七. 收获

1. 面向对象编程思想

通过抽象基类 Entity 和 State 实现多态，将游戏拆分为独立模块，深刻理解继承、封装、多态的核心作用，代码结构更清晰，便于维护与扩展。

2. 游戏开发核心技术

掌握 2D 游戏开发流程，包括玩家控制、碰撞检测、状态管理、特效渲染、音效播放等关键技术，理解 SFML 图形库的使用方法。

3. 工程化与项目管理

学会使用 GitHub/Gitee 进行代码托管、版本管理，理解工程化开发流程，能够规范管理项目文件与资源。

4. 资源制作与全流程开发

独立完成从技术选型、代码编写、美术资源设计到程序调试、测试的完整游戏开发流程，具备全栈开发思维。

5. 问题排查与调试能力

在测试中发现碰撞失效、生成频率异常、资源加载失败等问题，学会通过日志、断点调试定位并修复 bug，提升了程序调试与问题解决能力。

6. AI 工具的合理使用

AI 工具能快速提供学习资料、开发思路、代码示例和美术方案，大幅提升开发效率，但必须结合自身知识验证代码逻辑，培养了独立思考与甄别优化的能力。